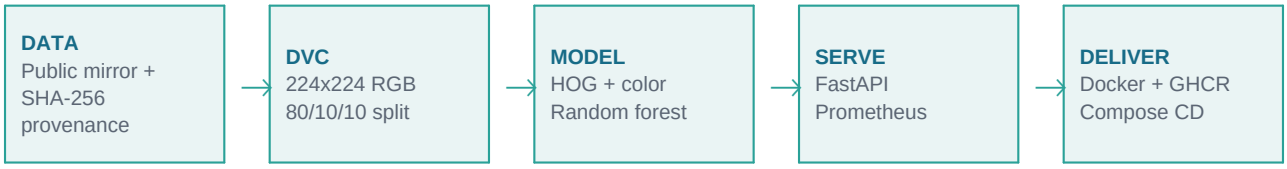
ASSIGNMENT 2

End-to-End MLOps Pipeline

Binary Image Classification: Cats vs Dogs



A complete, reproducible implementation for model development, artifact and image creation, packaging, containerization, CI/CD deployment, monitoring, and post-deployment evaluation.

Course	MLOps (S1-25_AIMLCZG523)
Use case	Pet adoption platform - Cats vs Dogs
Submission	Source, DVC, model, Docker, CI/CD, report, demo
Prepared	20 August 2026

1. Executive Summary

This project implements all five assignment modules as one executable repository. A balanced 600-image sample from the Microsoft/Kaggle Cats-vs-Dogs collection is downloaded through a public mirror with per-image SHA-256 provenance. DVC produces deterministic 80/10/10 splits and runs training. MLflow logs parameters, evaluation metrics, the serialized model, confusion matrix, sample requests, and learning curves.

The inference layer is a FastAPI service with health, prediction, and Prometheus metrics endpoints. Docker and Docker Compose define packaging and deployment. GitHub Actions tests every change, builds an immutable image, publishes to GHCR, deploys main to a Compose host over SSH, and fails the release if smoke tests do not pass.

Verified result: 70% test accuracy, 0.719 F1, and 5/5 automated tests passing.

Assignment-to-deliverable map

Module	Implemented evidence	Key files
M1	Git/DVC, 224x224 preprocessing, model, MLflow	dvc.yaml, params.yaml, train.py, model.joblib
M2	FastAPI, pinned environment, Docker	api.py, requirements.txt, Dockerfile
M3	pytest, GitHub Actions, GHCR publish	tests/, .github/workflows/mlops.yml
M4	Compose target, SSH CD, smoke tests	docker-compose.yml, smoke_test.py
M5	Structured logs, counters/latency, labeled batch	api.py, sample_requests.csv

Technology choices

Concern	Choice	Reason
Versioning	Git + DVC	Separates lightweight code history from data/model lineage.
Tracking	MLflow	Open source, local-first, logs metrics and arbitrary artifacts.
Model	Random forest	Fast laptop baseline with calibrated class proportions.
Serving	FastAPI	Typed REST API, validation, and automatic OpenAPI docs.
Delivery	Docker + GHCR + Compose	Portable artifact and a simple reproducible target.

2. M1 - Model Development and Experiment Tracking

2.1 Data and code versioning

Git versions all source and configuration files. DVC defines two dependency-aware stages: **prepare** and **train**. The lock file captures hashes for code, data, parameters, processed output, metrics, history, and the trained model. A local DVC remote is configured for offline reproducibility; it can be replaced with S3 or another remote without changing the pipeline.

```
$ python scripts/download_data.py --output data/raw
$ dvc repro
Running stage 'prepare' ... Prepared 600 images
Running stage 'train' ... metrics + model + MLflow run
```

2.2 Dataset and preprocessing

Property	Value
Source	Microsoft Cats vs Dogs, public Hugging Face mirror
Selected images	600 total: 300 cats, 300 dogs
Preprocessing	EXIF orientation, RGB, center crop, 224x224; train horizontal flips
Split	480 train / 60 validation / 60 test (80% / 10% / 10%)
Integrity	Source row and SHA-256 stored for every downloaded image
Reproducibility	Seed 42; deterministic sampling and class-balanced split

2.3 Baseline model

Every standardized image is summarized by HOG-style edge descriptors, a compact 16x16 RGB thumbnail, and per-channel color histograms. A 300-tree random forest is grown in 12 increments of 25 trees; the increments provide train/validation learning curves. Original-only and horizontally augmented candidates are compared using validation accuracy (0.733 vs 0.667), so the original-only candidate is selected without consulting the test set. The final estimator supports class probabilities. The complete inference bundle is serialized with joblib and includes feature settings, class names, model version, and evaluation metadata. Deterministic horizontal flips expand 480 training images into 960 training samples without contaminating validation or test data.

2.4 MLflow tracking

- Experiment: **cats-vs-dogs-baseline**; run: **random-forest-hog-baseline**.
- Parameters: feature size, histogram bins, epochs, trees per epoch, max features, seed.
- Metrics: accuracy, log loss, precision, recall, and F1.
- Artifacts: model.joblib, metrics JSON, confusion matrix, CSV history, curves, labeled request batch.

3. Evaluation Results

Metric	Result	Interpretation
Accuracy	0.700	Overall correctness on 60 held-out images
Precision (dog)	0.676	Purity of the predicted-dog class
Recall (dog)	0.767	Coverage of actual dog images
F1 (dog)	0.719	Harmonic balance of precision and recall
Log loss	0.620	Probability-quality metric; lower is better

Confusion matrix

Actual / Predicted	Cat	Dog
Cat	19	11
Dog	7	23

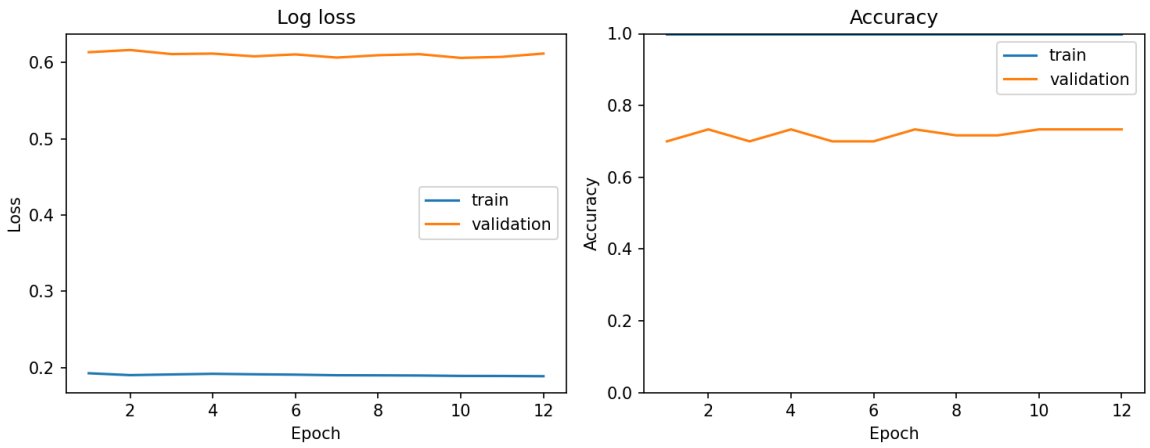


Figure 1. Training and validation loss/accuracy as the forest grows from 25 to 300 trees. The gap indicates expected overfitting for a deliberately small baseline dataset; transfer learning on the full dataset is the recommended next model iteration.

Post-deployment performance batch

The training pipeline writes 20 simulated production requests with image identifier, true label, predicted label, dog probability, and correctness. This establishes the schema for delayed-ground-truth monitoring without logging image bytes or personal information.

4. M2 - Packaging and Containerization

4.1 Inference API

Method	Endpoint	Behavior
GET	/health	Returns service status and loaded model version
POST	/predict	Accepts JPEG/PNG/WebP up to 10 MB; returns label and probabilities
GET	/metrics	Exports Prometheus request count and latency series
GET	/docs	Interactive OpenAPI documentation generated by FastAPI

```
POST /predict
{
  "label": "dog",
  "confidence": 0.5267,
  "probabilities": {"cat": 0.4733, "dog": 0.5267},
  "model_version": "1.0.0"
}
```

4.2 Reproducible environment

Production and development dependencies are fully pinned. The package uses a src layout and declares Python 3.11+. The service validates content type, file size, and decoded image content, then executes exactly the same feature path used during training.

4.3 Container

- Base: **python:3.11-slim**; no compiler or notebook runtime in the final image.
- Runs as a non-root system user and exposes only port 8000.
- Includes a Docker health check against **/health**.
- Copies the versioned model artifact into a fixed, environment-overridable path.
- A .dockerignore excludes datasets, caches, tests, reports, and local environments.

```
$ docker build -t cats-dogs-mlops:local .
$ IMAGE_NAME=cats-dogs-mlops:local docker compose up -d
$ python scripts/smoke_test.py --image sample.jpg
```

5. M3 - Continuous Integration

The GitHub Actions workflow triggers on all pushes and pull requests to main. Jobs use least-privilege package permissions and immutable commit-SHA image tags.

Step	Automated gate	Failure effect
Checkout/setup	Python 3.11 and pip cache	Workflow stops
Install	Pinned prod + dev dependencies	Workflow stops
Test	Five pytest unit/API tests	Image is not built
Build	BuildKit multi-platform-capable build	Image is not published
Publish	GHCR login with GITHUB_TOKEN	Deployment is blocked
Tag	Commit SHA and latest	Supports rollback and convenience

Automated test coverage

- Grayscale-to-RGB conversion, 224x224 output shape, normalized float array.
- Invalid image rejection.
- Feature vector and prediction response contract.
- Health and valid multipart prediction endpoints.
- Unsupported upload content type returns HTTP 415.

```
$ pytest -q
.....  [100%]
5 passed in 1.96s
```

Artifact publishing

On pushes, GitHub Actions authenticates to **ghcr.io** with the repository-scoped token and publishes both **ghcr.io/<owner>/cats-dogs-mlops:<commit-sha>** and **:latest**. Pull requests build but do not push, preventing untrusted publication.

6. M4 - Continuous Deployment

The selected target is Docker Compose on a simple VM. A production GitHub environment can enforce approval and secret scoping. After a successful main-branch image publish, the deploy job loads an SSH key, connects to the host, pulls the immutable SHA tag, and runs **docker compose up -d --remove-orphans**.

```
push to main -> tests -> build -> GHCR push -> SSH deploy -> health + prediction smoke test
failure anywhere ------------------------------------------------> release fails
```

Required deployment configuration

Name	Type	Purpose
DEPLOY_HOST	Secret	Docker Compose host DNS name or IP
DEPLOY_USER	Secret	Least-privilege SSH account
DEPLOY_SSH_KEY	Secret	Private deployment key
DEPLOY_PATH	Variable	Host checkout path; defaults to /opt/cats-dogs-mlops

Smoke test

The smoke script retries health during startup, submits one multipart image, and requires HTTP 200 with a label and probability map. Any failure exits non-zero and fails deployment.

7. M5 - Monitoring, Logging, and Performance

Request/response logging

Middleware emits request ID, HTTP method, route, status, and latency. Prediction logs contain only label, confidence, media type, and byte count. Filenames, image contents, and request bodies are excluded to avoid leaking user data.

```
INFO request id=... method=POST path=/predict status=200 latency_ms=48.21
INFO prediction label=dog confidence=0.5267 content_type=image/jpeg bytes=21989
```

Prometheus metrics

Metric	Labels	Use
inference_requests_total	endpoint, status	Traffic and error-rate monitoring
inference_latency_seconds	endpoint	Latency distribution and SLO tracking
process_* / python_*	standard	Runtime resource and garbage-collection signals

Performance feedback loop

The provided labeled-request CSV can be appended with production predictions and delayed true labels. A scheduled job can compute rolling accuracy/F1, compare them with the baseline, and trigger investigation or retraining when quality falls below an agreed threshold. Confidence histograms and input feature statistics can provide early drift signals even before labels arrive.

Operational recommendations

- Alert when 5xx rate exceeds 1% for 5 minutes or p95 latency exceeds the SLO.
- Retain only non-sensitive structured logs and apply a bounded retention policy.
- Evaluate quality by source/channel to expose sampling bias.
- Promote a new model only after offline metrics and shadow traffic checks pass.

8. Verification Evidence

Check	Observed result	Status
DVC reproduce	600 images, two stages, lock file updated	PASS
Model artifact	models/model.joblib, 516 KB	PASS
MLflow	Runs, params, metrics, and artifacts written	PASS
pytest	5 passed, 0 warnings	PASS
Live API	Health + prediction HTTP 200	PASS
Monitoring	Request counters and latency exported	PASS
Compose manifest	Docker Compose configuration included	PASS
Container pull	Docker Hub base-image access required	Environment dependent

9. Reproduction and Demonstration

Reproduce training

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt -r requirements-dev.txt
pip install .
python scripts/download_data.py --output data/raw
dvc repro
pytest -q
```

Run and inspect

```
uvicorn mlops_cats_dogs.api:app --host 0.0.0.0 --port 8000
curl http://localhost:8000/health
curl -X POST -F 'file=@sample.jpg' http://localhost:8000/predict
curl http://localhost:8000/metrics
mlflow ui --backend-store-uri ./mlruns
```

Submission contents

Artifact	Included
Source + tests + scripts	Yes
DVC pipeline and lock	Yes
Trained model and evaluation artifacts	Yes
Dockerfile + Compose	Yes
CI/CD workflow	Yes
PDF report	Yes
Sub-five-minute demonstration video	Yes

Limitations and next steps

This is a deliberately lightweight baseline trained on 600 images, not a production pet recognition model. Accuracy should be improved through transfer learning (for example, MobileNet or ResNet), a larger stratified dataset, augmentation, calibration, and model fairness checks. The CI/CD configuration is complete but requires repository secrets, a GHCR namespace, and an external Compose host to execute the production deployment path.

References

- Kaggle, *Dogs vs Cats competition dataset*: https://www.kaggle.com/competitions/dogs-vs-cats/data
- TensorFlow Datasets, *cats_vs_dogs catalog*: https://www.tensorflow.org/datasets/catalog/cats_vs_dogs
- DVC documentation: https://dvc.org/doc
- MLflow tracking documentation: https://mlflow.org/docs/latest/ml/tracking/
- FastAPI documentation: https://fastapi.tiangolo.com/
- GitHub Container Registry: https://docs.github.com/packages/working-with-a-github-packages-registry/working-with-the-container-registry
- Prometheus Python client: https://prometheus.github.io/client_python/